

Java Backend Architecture

Interview Series

Chapter 14.9

System Design

Walkthroughs Part 1

50+ Interview Q&A

Code Examples

Architecture Diagrams

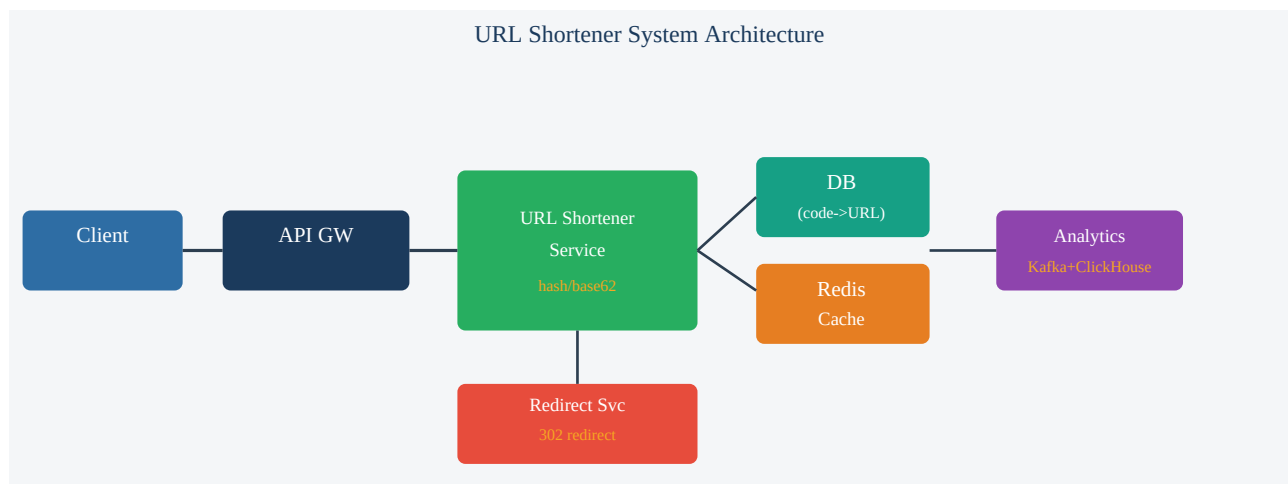
Advanced Topics

Chapter 14.9 - System Design Walkthroughs Part 1

Overview

This chapter walks through four classic system design problems: URL Shortener, Rate Limiter, Web Crawler, and Notification System. Each demonstrates core distributed systems principles including hashing, idempotency, deduplication, and fan-out patterns.

1. URL Shortener



Core Design Decisions

Short code generation	hash(longURL) -> truncate to 6 chars (base62: a-z, A-Z, 0-9 = $62^6 = 56$ billion combinations). Or counter-based: auto-increment ID converted to base62.
Collision handling	For hash approach: check DB for collision, append random suffix and retry. For counter approach: no collision - each ID is unique by construction.
Redirect type	302 Found (temporary): every request goes through service - enables click tracking, A/B testing, expiry enforcement. 301 (permanent): browser caches - no tracking possible.
Analytics	Log click events to Kafka; ClickHouse aggregates for real-time stats (clicks per hour, country, device). Decoupled from the hot redirect path.
Cache	Redis: key=short_code, value=long_url. 80% of traffic hits 20% of URLs (hot links). Cache TTL matches URL expiry. Cache-aside pattern.
DB schema	TABLE urls(code VARCHAR(8) PK, long_url TEXT, user_id, created_at, expires_at, click_count)

URL Shortener Key Calculations

- QPS: 100M URLs created/day = 1,160 writes/sec; reads 10x = 11,600 reads/sec
- Storage: 100M * 500 bytes avg = 50 GB/day; 5-year = 91 TB
- base62 with 6 chars: $62^6 = 56.8$ billion unique codes (sufficient for decades)
- Read path: cache-aside - check Redis first (cache hit ~80%), then DB

URL Shortener Code

```
@Service public class UrlShortenerService { private static final String BASE62 =
"abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"; // Counter-based (no
collision) public String shortenUrl(String longUrl) { long id = idGenerator.nextId(); //
Snowflake or DB sequence String code = toBase62(id); urlRepository.save(new UrlMapping(code,
longUrl, Instant.now())); redisTemplate.opsForValue().set(code, longUrl, 7, TimeUnit.DAYS);
return "https://short.ly/" + code; } public String toBase62(long num) { StringBuilder sb = new
StringBuilder(); while (num > 0) { sb.append(BASE62.charAt((int)(num % 62))); num /= 62; }
return sb.reverse().toString(); } // Redirect with caching public String getLongUrl(String
code) { String cached = redisTemplate.opsForValue().get(code); if (cached != null) return
cached; String url = urlRepository.findByCode(code) .orElseThrow(() -> new
NotFoundException("Code not found: " + code)); redisTemplate.opsForValue().set(code, url, 7,
TimeUnit.DAYS); return url; } }
```

2. Rate Limiter Design

Rate Limiter: Token Bucket vs Sliding Window Log

TOKEN BUCKET

Capacity: 10 tokens
Refill: 2 tokens/sec



Tokens: 4/10 (burst to 10)

SLIDING WINDOW LOG

Redis Sorted Set per user
Score = timestamp
ZADD + ZCOUNT in window
ZREMRANGEBYSCORE (expire old)
Exact but $O(\log N)$ + memory

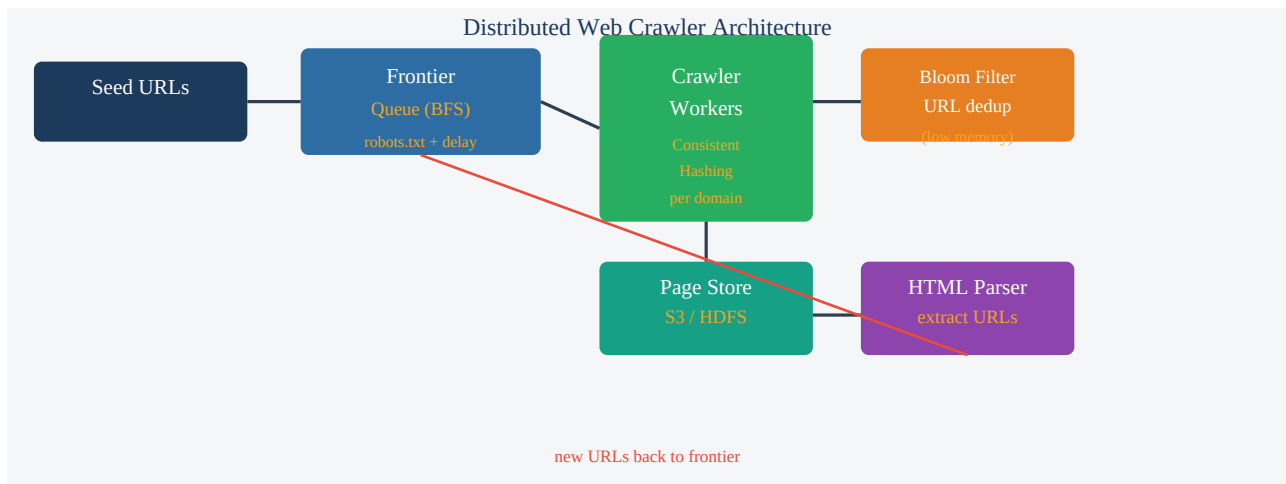
Rate Limiter Algorithms Comparison

Algorithm	Storage	Accuracy	Burst	Complexity
Token Bucket	Redis: last_refill + tokens	Good	Yes, up to capacity	Low
Leaky Bucket	Queue per user	Exact	No (smooth output)	Medium
Fixed Window	Redis: counter + window key	Poor (boundary spikes)	Yes (2x at boundary)	Very Low
Sliding Window Log	Redis sorted set (timestamp)	Exact	No	High (memory $O(N)$)
Sliding Window Counter	Redis: 2 counters	Approx (5% error)	Limited	Low

Redis Token Bucket Implementation

```
// Token bucket rate limiter with Redis (Lua script for atomicity) public boolean
allowRequest(String userId) { String key = "ratelimit:" + userId; // Lua ensures atomicity
(read-modify-write) String luaScript = "local tokens = tonumber(redis.call('get', KEYS[1]) or
ARGV[1])\n" + "local last = tonumber(redis.call('get', KEYS[2]) or ARGV[3])\n" + "local now =
tonumber(ARGV[3])\n" + "local refill = math.floor((now - last) / 1000 * tonumber(ARGV[2]))\n" +
"tokens = math.min(tonumber(ARGV[1]), tokens + refill)\n" + "if tokens >= 1 then\n" + "
redis.call('set', KEYS[1], tokens - 1)\n" + " redis.call('set', KEYS[2], now)\n" + " return
1\n" + "else return 0 end"; Long result = redisTemplate.execute( new
DefaultRedisScript<>(luaScript, Long.class), List.of(key + ":tokens", key + ":last"), "10", //
capacity "2", // refill rate per sec String.valueOf(System.currentTimeMillis()) ); return
result != null && result == 1L; }
```

3. Distributed Web Crawler



Web Crawler Design Components

Frontier Queue	Priority queue of URLs to crawl. BFS traversal: seed URLs added first, extracted URLs added with priority. Persistent (Kafka or DB-backed) for resumability.
Politeness	Respect robots.txt (fetch and cache per domain). Add crawl delay between requests to the same domain. Consistent hashing assigns domains to specific workers - one worker per domain maintains delay.
Deduplication	Bloom filter checks if URL was already seen before adding to frontier (false positives acceptable - skip some new URLs). Redis set for exact dedup on smaller crawls.
HTML Parser	Extracts links from crawled pages. Normalize URLs (remove fragments, canonicalize). Filter out non-HTML (images, PDFs). Add new URLs to frontier.
Page Storage	Crawled content stored in S3/HDFS with metadata (URL, crawl timestamp, HTTP status, content hash for change detection).
Consistent Hashing	URL domain is hashed to assign to a specific crawler worker node. Ensures only one worker crawls a given domain (maintains politeness delay, avoids duplicate crawl).

Bloom Filter for URL Deduplication

- Bloom filter: probabilistic data structure, $O(1)$ insert and lookup
- False positives possible (may skip a URL); false negatives impossible (never marks seen URL as new)
- Space: 1 billion URLs, 1% false positive rate needs ~1.2 GB (vs 50 GB for a HashSet)
- Java: Guava BloomFilter or RedisBloom module for distributed crawlers

```
// Guava Bloom Filter BloomFilter<String> seen = BloomFilter.create(
Funnel<String>.of(StandardCharsets.UTF_8), 1_000_000_000L, // expected insertions 0.01 //
1% false positive rate ); public boolean shouldCrawl(String url) { if (seen.mightContain(url))
return false; // might be seen seen.put(url); return true; }
```

4. Notification System Design

Notification System Architecture

Channels	Push notifications (FCM for Android, APNs for iOS), Email (AWS SES, SendGrid), SMS (Twilio), In-app (WebSocket).
Fan-out	For events affecting many users (new follower, marketing campaign): publish event to Kafka; notification workers consume and send to each user's preferred channels.
User Preferences	Users configure notification settings (which events, which channels, quiet hours). Check preferences before sending - respect opt-outs.
Rate limiting	Limit notifications per user per day (e.g., max 10 push per day per app) to avoid spamming. Track count in Redis.
Delivery tracking	Track delivery status: PENDING -> SENT -> DELIVERED -> READ. Store in notifications table. Support retry on failure (dead letter queue for failed sends).
Priority queues	Critical alerts (security, OTP) on high-priority queue; marketing on low-priority. High-priority skips rate limits.

Notification Fan-out Flow

```
// Notification Service @Service public class NotificationService { public void
sendNotification(NotificationEvent event) { // Get user preferences UserPrefs prefs =
prefsService.get(event.getUserId()); if (!prefs.isEnabled(event.getType())) return; // Check
rate limit if (!rateLimiter.allowNotification(event.getUserId())) { log.info("Rate limited
userId={}", event.getUserId()); return; } // Fan-out to channels if (prefs.isPushEnabled() &&
event.getUserId() != null) { pushService.send(event.getUserId(), event); } if
(prefs.isEmailEnabled() && prefs.getEmail() != null) { emailService.send(prefs.getEmail(),
event); } if (prefs.isSmsEnabled() && prefs.getPhone() != null) {
smsService.send(prefs.getPhone(), event); } } } // Push notification via FCM @Service public
class PushService { public void send(String userId, NotificationEvent event) { List<String>
deviceTokens = deviceRepo.findByUserId(userId); for (String token : deviceTokens) {
fcmClient.send(FcmMessage.builder() .token(token) .title(event.getTitle())
.body(event.getBody()) .build()); } } }
```

Interview Q&A - System Design Walkthroughs Part 1

Q1. Design a URL shortener. What are the key components?

Key components: (1) Short code generation - counter-based (Snowflake ID to base62, no collision) or hash-based (MD5/SHA-256, truncated, with collision check). (2) Database: urls table (code, long_url, user_id, created_at, expires_at). (3) Redis cache for hot URLs (80% hit rate expected). (4) Redirect service returning 302. (5) Analytics pipeline: Kafka events -> ClickHouse. (6) API: POST /shorten, GET /{code}.

Q2. Why use 302 over 301 for URL shortener redirects?

301 Moved Permanently is cached permanently by browsers - subsequent requests bypass the service entirely, preventing click analytics, A/B testing, and real-time expiry enforcement. 302 Found is temporary - every click goes through the service, enabling tracking (IP, device, referer), A/B routing, and honoring URL expiration. The latency cost (~10-50ms for the redirect) is acceptable for analytics value.

Q3. Explain base62 encoding for URL shortening.

Base62 uses 62 characters: a-z (26) + A-Z (26) + 0-9 (10). A 6-character code has $62^6 = 56.8$ billion combinations. Convert a numeric counter (Snowflake ID) to base62 by repeatedly dividing by 62 and mapping remainders to characters. Benefits over hash: no collisions (each ID maps to unique code), sequential IDs produce short codes, no extra DB lookup to check for collision.

Q4. How do you handle hash collisions in URL shortening?

When hashing the long URL, take the first 6 characters of MD5/SHA-256. Check if the code already exists in DB. If yes and maps to a different URL (collision): append a random suffix to the long URL and rehash, repeat until no collision. Alternatively, pre-check multiple hash variations. Counter-based approach eliminates this entirely - each increment produces a unique code.

Q5. How do you scale the URL shortener to 10,000 reads/sec?

Redis cache in front of the DB: cache hot short codes (80% of reads hit 20% of codes, cache-aside). Horizontal scale the redirect service (stateless, add nodes behind a load balancer). Read replicas for the DB (redirect service reads from replica). CDN for the redirect response if 301 is acceptable (for very static links). Globally distributed Redis (Elasticache) for sub-millisecond cache lookups worldwide.

Q6. What is a token bucket rate limiter?

The token bucket algorithm: a bucket holds up to capacity tokens. Tokens refill at rate r /second. Each request consumes 1 token. If the bucket is empty, the request is rejected (HTTP 429). Key property: burst traffic up to the bucket capacity is allowed. Over the long term, the average rate is bounded by r . Stored in Redis: two keys per user - token count and last refill timestamp.

Q7. How does sliding window log rate limiting work with Redis?

Use a Redis sorted set per user (key = "ratelimit:{userId}"). Score = request timestamp (milliseconds). For each request: ZREMRANGEBYSCORE to remove timestamps older than the window (now - window_ms); ZCARD to count remaining entries; if count < limit: ZADD current timestamp and allow; else reject. EXPIRE on the key for cleanup. Exact accuracy, $O(\log N)$ per request, memory $O(\text{requests_in_window})$.

Q8. What is the fixed window rate limiting boundary spike problem?

Fixed window divides time into discrete buckets (e.g., 1-minute windows). A client can make max_requests at 11:59:59 and max_requests again at 12:00:00 - effectively 2x the limit in 2 seconds. The sliding window

approaches fix this by considering a rolling window from the current time, but require more Redis operations. Sliding window counter approximates using linear interpolation between windows to avoid the spike.

Q9. How does sliding window counter rate limiting work?

Maintain two counters: current window count and previous window count. The current count is interpolated: $\text{effective_count} = \text{prev_count} * (\text{window_size} - \text{elapsed}) / \text{window_size} + \text{curr_count}$. This approximates the sliding window count with two Redis integers instead of a sorted set. Slightly less accurate (max 5% error at window boundaries) but $O(1)$ Redis operations vs $O(\log N)$ for sorted set.

Q10. What is distributed rate limiting and how do you implement it?

In a multi-node service (load-balanced API), each node must share rate limit state. Use Redis as a centralized atomic counter. Lua scripts ensure read-modify-write atomicity. Alternative: sticky sessions (same user always routes to same node) - simple but breaks on node failure. Redis Cluster with per-user key routing ensures low latency. For very high rates, consider token bucket with local approximate counters + Redis sync.

Q11. Design a web crawler. What are the key challenges?

Key challenges: (1) Scale: web has 5+ billion pages - need distributed crawling. (2) Politeness: respect robots.txt and crawl delays to avoid overloading sites. (3) Deduplication: avoid recrawling the same URL (Bloom filter for efficiency). (4) URL frontier management: BFS with priority queue, persistent for resumability. (5) Dynamic content: JavaScript-rendered pages (headless browser). (6) Freshness: re-crawl changed pages on schedule.

Q12. How does consistent hashing help in a distributed web crawler?

Consistent hashing assigns URL domains to specific crawler worker nodes. All URLs from example.com route to the same worker. Benefits: (1) Politeness: the worker controls the crawl delay for that domain without distributed coordination; (2) Deduplication: domain-level state is local to one worker; (3) No race conditions: only one worker crawls a domain simultaneously. When a worker is added/removed, only its adjacent domains reassign.

Q13. What is a Bloom filter and how is it used in a web crawler?

A Bloom filter is a probabilistic, space-efficient data structure for set membership. It never has false negatives (if it says "not seen", definitely not seen) but may have false positives (says "seen" when it was not). For URL deduplication: before adding a URL to the frontier, check the Bloom filter. If "seen" (may be false positive), skip. If "not seen", add. 1 billion URLs at 1% FPR needs only ~1.2 GB - much cheaper than a HashSet (~50 GB).

Q14. How do you handle robots.txt in a web crawler?

Fetch robots.txt for each domain before crawling any URL on that domain (cache the robots.txt with TTL). Parse Disallow rules for your User-Agent. Skip URLs that match Disallow patterns. Respect Crawl-delay directive (wait N seconds between requests to the same domain). Store robots.txt cache in Redis (domain -> rules) with TTL matching the Crawl-delay or 24h default. Re-fetch if cache expires.

Q15. How do you handle duplicate content in a web crawler?

URL-level dedup: Bloom filter or Redis set prevents crawling the same URL twice. Content-level dedup: compute hash (MD5/SHA-256) of crawled content; skip storing if hash already seen (different URL, same content - e.g., www vs non-www). SimHash for near-duplicate detection (detect pages with 90% similar content). Store canonical URLs (strip tracking parameters) to reduce URL variants.

Q16. Design a notification system. What are the main components?

Components: (1) Event bus (Kafka): services publish notification events. (2) Notification Service: consumes events, checks user preferences, dispatches to channel workers. (3) Channel workers: Push (FCM/APNs), Email (SES), SMS (Twilio) with retry logic. (4) User preferences DB: opt-in/opt-out per channel/event type, quiet hours. (5) Device token registry: maps user to device tokens for push. (6) Notification history: tracks delivery status.

Q17. What is fan-out in a notification system and when is it expensive?

Fan-out sends a single event to multiple recipients. Example: posting a tweet notifies all followers. For a user with 1,000 followers, fan-out is 1,000 push notifications. For a celebrity with 50 million followers, fan-out is extremely expensive (50M DB reads + 50M queue messages). Solutions: batch fan-out (process in chunks), async fan-out (Kafka consumers), lazy fan-out (pull notifications at read time for very large fan-outs), or hybrid based on follower count threshold.

Q18. How does FCM (Firebase Cloud Messaging) work?

FCM: app registers with FCM and receives a device token (unique per app-device combo). Your server stores this token. To send a push: call FCM API with the device token and notification payload. FCM delivers to the device via persistent connection (Android) or APNs bridge (iOS). FCM handles: retry on temporary failures, message queuing (if device offline, deliver on reconnect), multicast (send to multiple tokens in one call), topic subscriptions.

Q19. How do you handle notification delivery failures?

Retry with exponential backoff for transient failures (HTTP 500, network timeout). Dead letter queue (DLQ) for messages that fail after max retries - alert on DLQ depth. For push: invalid token (HTTP 404) means token is expired/revoked - delete from device registry. For email: handle bounces (hard bounce = invalid address, remove from list; soft bounce = temporary, retry). Track delivery status in a notifications table (PENDING, SENT, FAILED, DELIVERED).

Q20. How do you prevent notification spam?

Per-user rate limits: max N notifications per channel per day (track in Redis with expiring counters). Notification batching: group similar notifications (5 new likes -> "X and 4 others liked your post"). Quiet hours: do not send push/SMS outside user's configured hours (delay or drop). Unsubscribe/opt-out: one-click unsubscribe in emails (required by CAN-SPAM). Priority levels: marketing notifications can be delayed or batched; critical alerts bypass rate limits.

Q21. What is the difference between push notifications and pull notifications?

Push: server initiates delivery to client when event occurs (FCM, APNs, WebSocket). Low latency, real-time updates, battery drain, requires persistent connection or push service. Pull: client periodically polls server for new notifications (long-polling, SSE). Simpler server-side (no push infrastructure), but latency = poll interval, wasteful if no updates. Hybrid: push for real-time (mobile apps), pull fallback for web clients.

Q22. How do you design for idempotency in a notification system?

Assign a unique notificationId to each notification event. Before sending, check if this notificationId was already processed (Redis set or DB record). If yes, skip (idempotent). This handles Kafka redelivery (at-least-once semantics). For email: include a Message-ID header for deduplication at the SMTP level. For SMS: provider idempotency keys (Twilio DeduplicationId) prevent duplicate sends on retry.

Q23. How do you handle time zones in a notification system?

Store user's timezone (IANA timezone string, e.g., "America/New_York"). For quiet hours: compare current time in user's timezone against configured quiet hours. For scheduled notifications: convert scheduled time from user's timezone to UTC for storage. Java: `ZonedDateTime.of(user.getTimezone(), ZonedDateTime.now(zoneId))`. For recurring notifications (daily digest): calculate next send time in user's timezone.

Q24. What is APNs and how is it different from FCM?

APNs (Apple Push Notification Service) is Apple's push notification infrastructure for iOS/macOS. FCM (Firebase Cloud Messaging) is Google's for Android, and also routes to APNs for iOS via Firebase. Differences: APNs requires HTTP/2 with JWT authentication; FCM uses HTTP API with service account key. APNs is required for iOS delivery (cannot use FCM directly to iOS without Firebase relay). Use Firebase Unified SDK to handle both platforms via one API.

Q25. How do you design URL expiry for a URL shortener?

Store `expires_at` timestamp in the DB. Redirect service: on lookup, check if `expires_at < now` -> return HTTP 410 Gone. Cleanup: background job (cron or Spring `@Scheduled`) runs daily to DELETE expired URLs from DB and Redis. For high-volume: use a batch job with soft delete (mark as expired) then async purge. For Redis: set TTL on cache keys matching the URL expiry, so cache auto-expires without cleanup job for Redis.

Q26. What is a custom short code and how do you handle it?

Custom codes allow users to choose their short URL (e.g., `short.ly/my-brand`). Design: attempt INSERT with user-provided code; catch `UniqueConstraintException` to handle conflicts. Validate code: alphanumeric, 3-20 chars, not a reserved word (api, admin, health). Rate limit custom code creation per user. Custom codes should be case-sensitive or consistently normalized (lowercase) to avoid confusion.

Q27. How do you design a rate limiter for an API with multiple tiers?

Define tiers: Free (100 req/min), Pro (1,000 req/min), Enterprise (10,000 req/min). Store tier in Redis keyed by API key or user ID. Rate limiter reads tier config to determine limit. Implement as middleware: check tier limit before processing request; return HTTP 429 with `Retry-After` header and `X-RateLimit-Remaining` header for client guidance. Separate limits per endpoint (read vs write operations may have different limits).

Q28. What is a priority queue in the context of a web crawler frontier?

The crawler frontier prioritizes URLs by importance: PageRank or link count (important pages crawled first), freshness (frequently updated pages crawled more often), domain priority (partner sites first). Implemented as a multi-bucket queue: high/medium/low priority buckets. The crawler dequeues from high-priority first. Kafka topics per priority level or a Redis sorted set with `score = priority * time`.

Q29. How do you handle JavaScript-rendered pages in a web crawler?

Standard HTTP crawlers cannot execute JavaScript; they see only the raw HTML. For JavaScript-heavy pages: use a headless browser (Playwright, Puppeteer, Selenium with headless Chrome). Render the page, wait for dynamic content, then extract the DOM. Much more expensive: 5-10x CPU and memory vs HTTP crawl. Strategy: detect if page requires JS (check for specific indicators), use headless browser only for those. Scale with a separate JS rendering cluster.

Q30. How do you design a notification preference system?

Schema: `user_notification_prefs`(`user_id`, `notification_type` ENUM, `channel` ENUM, `enabled` BOOL, `quiet_hours_start` TIME, `quiet_hours_end` TIME, `frequency` ENUM). Notification types: ORDER_UPDATE, MARKETING, SECURITY_ALERT, SOCIAL. Channels: PUSH, EMAIL, SMS. Override hierarchy: global (all channels on/off) > type-level > channel-level. Security alerts ignore quiet hours and rate limits. Cache

preferences in Redis per user (TTL 1h) for fast lookup on the hot notification path.

Q31. What is the role of Kafka in a notification system?

Kafka decouples event producers (Order Service publishes OrderShipped event) from notification consumers. Benefits: (1) Durability - events are not lost if notification service is down; (2) Scalability - multiple consumer instances process notifications in parallel; (3) Replay - reprocess missed notifications by resetting consumer offset; (4) Multiple consumers - notification service and analytics service both consume the same events. Topic per notification priority level.

Q32. How do you handle device token management for push notifications?

Device tokens can expire or change (user reinstalls app, token rotation). Design: store tokens in device_tokens table (user_id, platform ENUM, token, created_at, last_used_at). On FCM/APNs delivery: if HTTP 404 (invalid token), delete from table. On app launch: app refreshes token with server (UPSERT). User may have multiple devices - send to all tokens. Batch token validation against FCM/APNs periodically to prune stale tokens.

Q33. How do you handle analytics for a URL shortener at scale?

Write path: on each redirect, emit a click event to Kafka (short_code, timestamp, IP, user_agent, referer). Do not write to DB on the hot redirect path. Analytics consumers: Kafka Streams or Flink computes real-time aggregates per URL (clicks per minute, unique visitors). ClickHouse stores time-series click data for fast aggregation queries. Dashboard: Grafana or custom UI queries ClickHouse for charts (clicks over time, top countries, devices).

Q34. What is the Leaky Bucket rate limiting algorithm?

The leaky bucket processes requests at a constant fixed rate, like water leaking from a bucket at a steady pace. Incoming requests are added to a queue (bucket). If the bucket (queue) is full, requests are dropped. A background process drains the queue at the fixed rate. This smooths bursty traffic into a constant rate. Contrast with token bucket which allows bursts up to bucket capacity. Leaky bucket is ideal for traffic shaping to a downstream service.

Q35. How do you count unique visitors for URL analytics?

Exact count: Redis HyperLogLog (probabilistic, ~0.81% error, uses only 12KB per key). PFADD to add a visitor IP; PFCOUNT to get approximate unique count. Much more efficient than a SET which uses O(N) memory. For exact count: Bloom filter (can tell if seen before, mark as new/returning). For full analytics: ClickHouse with deduplication on (short_code, ip, date). HyperLogLog is the standard choice for scalable unique counting.

Q36. How do you make the web crawler resumable after failure?

Persist the frontier queue (not in memory): use Kafka topic as the frontier - URLs are durable messages. Commit consumer offsets only after a URL is crawled and stored. On restart, resume from last committed offset. Store crawl state in a DB: crawl_jobs(url, status ENUM(PENDING, IN_PROGRESS, DONE, FAILED), last_crawled_at, worker_id). On worker failure, requeue IN_PROGRESS jobs with elapsed time > timeout (cron-based recovery).

Q37. What is SMS delivery and how does Twilio work?

Twilio provides a REST API to send SMS: POST /Messages with To (recipient phone), From (Twilio number), Body (text, max 160 chars per segment). Twilio routes through carrier networks. For delivery receipt: configure StatusCallback URL - Twilio sends webhook (POST) with delivery status (sent, delivered, failed).

Multi-country compliance: some countries require local phone numbers. Long messages are split into segments (charged per segment, ~160 chars).

Q38. What is email deliverability and how do you avoid spam filters?

Key factors: (1) SPF record: DNS TXT listing authorized sending IPs. (2) DKIM: cryptographic signature on email headers. (3) DMARC: policy combining SPF+DKIM, instructs receivers how to handle failures. (4) Reputation: sending from a dedicated IP with good history. (5) Content: avoid spam trigger words, balance text/image ratio, include unsubscribe link. Use AWS SES or SendGrid which handle IP reputation management. Monitor bounce rate (<2%) and complaint rate (<0.1%).

Q39. How do you design URL analytics with time-series data?

Schema in ClickHouse: click_events(short_code, clicked_at DateTime, country String, device String, browser String) ENGINE=MergeTree PARTITION BY toYYYYMM(clicked_at) ORDER BY (short_code, clicked_at). Queries: SELECT count() FROM click_events WHERE short_code='abc123' AND clicked_at > now() - INTERVAL 7 DAY. ClickHouse handles billions of rows with sub-second analytics queries. Pre-aggregate into hourly summaries for dashboard performance.

Q40. What is the difference between horizontal and vertical scaling for a rate limiter?

Horizontal scaling (multiple nodes): each node must share state via Redis to enforce global limits - add Redis round-trip (~1ms) per request. Vertical scaling (bigger single node): all state local, no network hop, but single point of failure and finite ceiling. For most rate limiters, horizontal + Redis is the right choice. For extreme performance (100k+ req/sec), use local approximate counters with periodic Redis sync (accepting slight over-limit during sync interval).

Q41. How do you handle long URLs that exceed 2,048 character limits?

HTTP GET redirects work for most URLs but some proxies truncate long URLs. Solutions: (1) Store hash of long URL as lookup key (but need full URL for redirect). (2) Use POST for creation, store full URL in DB as TEXT (no length limit). (3) For redirect: return 301/302 with Location header - headers support long URLs (no 2,048 limit in spec, but some browsers/proxies impose limits). (4) Use URL encoding/compression for extremely long URLs.

Q42. What is a dead letter queue (DLQ) in notification systems?

A DLQ stores messages that failed processing after exhausting retries. In Kafka: configure max.delivery.count (Spring Kafka); after N failures, message is routed to a dead-letter topic (e.g., notifications.DLT). In SQS: built-in DLQ configuration. DLQ uses: (1) Investigate failures without losing messages; (2) Alert on DLQ depth (> 0 means processing failures); (3) Replay after fixing the bug: consume from DLQ back to main topic. Essential for production notification systems.

Q43. How do you handle URL shortener abuse (malicious URLs)?

Prevention: (1) Google Safe Browsing API: check long URL before shortening - reject malicious URLs. (2) Rate limit URL creation per user/IP. (3) Require authentication for creation (reduces anonymous abuse). Detection: (1) Re-scan existing URLs against Safe Browsing periodically. (2) User reports: "report this link" on the redirect page. (3) ML model on URL features (suspicious patterns). Response: mark as malicious (is_blocked=true), return 403 on redirect with warning.

Q44. How do you implement web crawler politeness at scale?

Per-domain rate limiting: use a Redis sorted set of domains with next_crawl_time as score. The scheduler dequeues domains whose next_crawl_time has passed. After crawling a URL from a domain, update

next_crawl_time = now + crawl_delay_ms (from robots.txt). Consistent hashing ensures one worker owns a domain, avoiding distributed coordination for delay enforcement. Global crawl rate: set max RPS budget per domain (e.g., 1 request/10s).

Q45. What is URL normalization in web crawling?

URL normalization reduces equivalent URLs to a canonical form to avoid duplicate crawls: lowercase scheme and host (HTTP://Example.COM -> http://example.com), default port removal (:80 -> empty), path normalization (/./, ../ resolution), URL decoding (hello%20world -> hello world, but keep necessary encoding), sort query params (?b=2&a=1 -> ?a=1&b=2), remove tracking params (?utm_source, ?fbclid), remove fragments (#section). Normalize before Bloom filter check.

Q46. How do you handle notification batching to reduce costs?

Batch notifications within a time window (e.g., 5 minutes) and aggregate similar events: "Alice, Bob, and 3 others liked your post" instead of 5 separate pushes. Implementation: queue notifications in Redis with TTL=300s; a scheduler collects pending notifications per user, aggregates by type, sends one batched message. Reduces push count 5-10x, improving cost and reducing user notification fatigue. High-priority events (OTP, security) bypass batching.

Q47. What is the capacity estimation process for a URL shortener?

Start with requirements: 100M new URLs/day. Write QPS: $100M / 86400 = 1,160/\text{sec}$. Read-to-write ratio: 10:1, so 11,600 reads/sec. Storage per URL: code(8B) + long_url(200B) + metadata(100B) = ~308B. Daily storage: $100M * 308B = 30.8\text{GB/day}$. 5-year storage: 56TB. Peak QPS: 3x average = ~35,000 reads/sec at peak. Redis memory for hot URLs: assume 10M hot URLs * 200B = 2GB. Throughput: $35,000 * 200B = 7\text{MB/s}$ bandwidth.

Q48. How do you design the database for a URL shortener to handle 100M records?

Schema: urls(id BIGINT PK, code VARCHAR(8) UNIQUE INDEX, long_url TEXT, user_id BIGINT INDEX, created_at TIMESTAMP, expires_at TIMESTAMP INDEX, is_active BOOL). Indexes: code (primary lookup - must be O(1) hash index), user_id (user's URLs list), expires_at (cleanup job). At 100M records, a single PostgreSQL instance handles this with proper indexing. Shard by user_id if needed beyond 1B records. Archive expired URLs to cold storage.

Q49. How do you ensure high availability for a notification system?

Kafka provides durability (replication factor 3) - events are not lost on service failure. Multiple notification service instances (scale horizontally, Kafka consumer group). Retry with DLQ for failed sends. Circuit breakers for external providers (FCM down -> fallback to email, or store in outbox for later retry). Health checks per channel: if email provider is down, route to backup (SES -> SendGrid fallback). Idempotency prevents duplicate sends on retry.

Q50. What is the Bloom filter false positive rate and how is it tuned?

False positive rate $p = (1 - e^{-(kn/m)})^k$ where k = number of hash functions, n = number of elements, m = number of bits. For $n=1B$ elements: $p=1\%$ needs $m=9.6$ bits/element = ~1.2GB, $k=7$ hash functions; $p=0.1\%$ needs ~14.4 bits/element = ~1.8GB, $k=10$. Reduce p by increasing m (more memory) or k (more hash functions, slower insertion). For web crawlers, 1% FPR is typically acceptable (we skip 1% of truly new URLs - minor quality loss vs enormous memory savings).

Q51. How do you handle in-app notification delivery with WebSockets?

On user login, establish a WebSocket connection to the notification server. The server maintains a map: `userId -> WebSocket session`. When a notification event arrives: if user has an active WebSocket, push immediately (real-time). If user is offline, store in notifications table (PENDING). On reconnect, send missed notifications. Scale: use Redis Pub/Sub - notification service publishes to `userId` channel, all WebSocket server instances subscribe, and the one holding the user's session delivers it.

Q52. What is the difference between transactional and promotional notifications?

Transactional notifications are triggered by user actions (order confirmation, password reset, OTP). High priority, time-sensitive, cannot be batch-delayed, exempt from marketing opt-outs. Required to be delivered. Promotional/marketing notifications are outbound campaigns (sale alerts, newsletters). Can be batched, rate-limited, must honor opt-outs, subject to CAN-SPAM/GDPR opt-in requirements. Separate sending queues, separate rate limits, separate opt-out management. Some email providers (SES) enforce separate IPs for transactional vs promotional.
